

Lab 3B: Subagents & Agent Teams

Duration: 35 min **After:** Subagents & Agent Teams lecture (Day 3, Block 2) **Prerequisites:** Lab 3A complete (MCP server configured)

Objective

Define a custom subagent, run a **nested delegation** (2+ levels deep), add a **grader-style review pass** against a rubric, and launch an **experimental agent team** — with a parallel-subagent fallback if the flag isn't available on your seat.

Deliverable (toolkit chain 10 of 12)

Two subagent definitions in `.claude/agents/` (a builder and a grader) plus a graded, nested multi-agent run you can explain.

START MARKER: `/clear` done in `business-dashboard` .

Fell behind? Run:

Set up the business-dashboard project with: CLAUDE.md conventions, an Express server with routes in `src/routes/`, and the dashboard-metrics MCP server in `.mcp.json`.

Phase 1: Define Custom Subagents (8 min)

Subagents are markdown files in `.claude/agents/` — isolated instances with their own context, tools, and (optionally) model.

Step 1 — a restricted auditor:

Create a subagent at `.claude/agents/security-auditor.md` in the current project directory:

```
---
name: security-auditor
description: Audit code for security vulnerabilities. Use when asked to "check security", "audit for vulnerabilities", or "review auth code".
tools: Read, Grep, Glob
---

# Security Auditor
Check for: hardcoded secrets, SQL injection, unvalidated input, eval(), unsafe deserialization.
Output: CRITICAL / WARNING / INFO findings with file and line references.
```

Step 2 — a grader with a rubric:

Create a subagent at `.claude/agents/grader.md`:

```
---
name: grader
description: Grade completed work against a rubric. Use when asked to "grade", "score against the rubric", or "run a review pass".
tools: Read, Grep, Glob
---

# Grader
Read the rubric file specified in the task. Score each criterion 0-2 (0=missing, 1=partial, 2=met).
Output a table: criterion | score | evidence (file:line). End with TOTAL: n/10 and a PASS (>=8) or REWORK verdict. Never fix code – only grade it.
```

Step 3 — verify:

`/agents`

Expected output marker: `security-auditor` and `grader` listed alongside built-ins.

Step 4 — first delegation:

Use the `security-auditor` subagent to audit everything in `src/` and save findings to `reports/security.md`

Expected output marker: the subagent runs in its own context, and `reports/security.md` appears with CRITICAL/WARNING/INFO findings. Current Claude Code backgrounds subagents by default; use the subagent panel to inspect status, transcript, tool calls, and usage while the main session remains responsive.

Phase 2: Nested Delegation — 2+ Levels (9 min)

Subagents can spawn subagents. The current default allows up to **three subagent layers below the main conversation** and can be changed with `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` (`1` disables nesting). You will run the shallow tree `you` → `coordinator` → `workers`.

```
Spawn a subagent acting as a coordinator with this task:
"Produce reports/quality-report.md for the src/ directory. Do NOT analyze the code yourself –
delegate: spawn one subagent to inventory all endpoints and their validation status, and a
second subagent to list every function missing error handling. Merge their two outputs into a
single report with sections: Endpoints, Error Handling Gaps, Top 3 Risks."
```

Watch the delegation tree: your session (level 0) → coordinator (level 1) → two workers (level 2).

Expected output marker: `reports/quality-report.md` exists with all three sections, and the transcript shows the coordinator spawning its own subagents — not doing the analysis itself.

```
Show me the delegation tree for that last task: who spawned whom, and what did each level contribute?
```

***When to nest:** the coordinator holds the MERGE logic; workers hold the DETAIL. Nesting keeps your main context clean — you receive one report, not two raw dumps. Don't nest for simple sequential work; every level multiplies token cost.*

Phase 3: Grader-Style Review Pass (8 min)

The Performance Outcomes pattern: work is not "done" until a separate grader scores it against an explicit rubric.

Step 1 — write the rubric:

```
Create rubric.md with 5 criteria for src/routes/tasks.js:
1. All inputs validated with guard clauses
2. try/catch on every route
3. Parameterized SQL only
4. Error responses use { error: "message" }
5. No console.log left in production paths
```

Step 2 — grade:

```
Use the grader subagent to score src/routes/tasks.js against rubric.md
```

Expected output marker: a criterion/score/evidence table ending in **TOTAL: n/10** and PASS or REWORK.

Step 3 — close the loop. If REWORK:

```
Fix only the criteria the grader scored below 2, then have the grader re-score.
```

Expected output marker: second grading run scores higher. Builder and grader are SEPARATE contexts — the grader can't be talked into leniency by the builder's reasoning. That separation is the point.

Phase 4: Agent Teams — EXPERIMENTAL (8 min)

***EXPERIMENTAL:** Agent Teams is disabled by default. Enable it with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`, review the current limitations, and keep the stable subagent fallback ready.*

Step 1 — try to enable:

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
claude
```

Then run:

```
Create an agent team to add a /api/export/summary endpoint:
- architect: design the endpoint and data shape
- implementer: build it following CLAUDE.md
- reviewer: grade the result against rubric.md
Architect first, then implementer, then reviewer.
```

Expected output marker: teammates spawn with their own contexts, shared task list, and peer messaging. They normally share the checkout, so the task must partition file ownership; worktrees are not automatic.

Step 2 — FALLBACK (flag unavailable — run this instead, same learning):

Run this as parallel subagents:

1. One subagent designs the `/api/export/summary` endpoint and writes the design to `reports/design.n.md`
 2. When the design exists, a second subagent implements it following `CLAUDE.md`
 3. The grader subagent then scores the implementation against `rubric.md`
- Coordinate the sequence yourself and show me each agent's summary.

Expected output marker: three agents run under your session's coordination — the difference from Teams is WHO coordinates (you/main agent vs the team runtime) and peer-to-peer messaging, which subagents don't have.

Step 3 — decision framework:

	Single session	Subagents (incl. nested)	Agent Teams
Coordination	you	main agent	team runtime (peer mailboxes)
Context	one window	isolated per agent	isolated per teammate; shared checkout unless you isolate it
Cost	lowest	per-agent multiplier	highest
Status	stable	stable	experimental and disabled by default

FINISH MARKER — Success Checklist

- [] Two subagents defined in `.claude/agents/` with restricted tools
- [] Nested delegation ran: you → coordinator → 2 workers (2+ levels)
- [] `reports/quality-report.md` merged by the coordinator, not by you
- [] Grader scored work against `rubric.md` with evidence and a verdict
- [] Rework loop raised the score
- [] Agent team launched OR fallback executed, and you can say which and why
- [] You can argue single vs subagents vs teams for a given task

If Stuck

Problem	Recovery
Subagent not found	Files must be in the PROJECT's <code>.claude/agents/</code> ; <code>/clear</code> to reload
Coordinator does the work itself	Re-prompt: "You must NOT analyze code directly — delegate to subagents and only merge"
Nested spawn refused	Check <code>CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH</code> ; flatten the design to a shallow coordinator + workers
Grader "fixes" the code	Its tools are read-only (Read/Grep/Glob) — if it edits, check the frontmatter tools line
Team does not start	Confirm the exact environment variable was set before launch; otherwise run the stable fallback
Token usage exploding	<code>/context</code> ; every agent level multiplies cost — narrow scopes

Debrief Questions

1. What did nesting buy you that two flat subagents wouldn't have?
2. Why must the grader be a separate agent instead of asking the builder "did you meet the rubric?"
3. Where's the line where you'd move from nested subagents to an agent team — and what would you verify first, given the feature is experimental?

